

Query-adaptive late segmentation: dynamic context assembly via sentence multi-vectors and split-conformal budgeting

Author 1 [Placeholder Author One]^a, Author 2 [Placeholder Author Two]^{b,*}

^a Department of Computer Science, University [Placeholder Institution A], Country
Email: author.one@institution.edu

^b Department of Information Systems, University [Placeholder Institution B], Country
Email: author.two@institution.edu

* Corresponding author. Tel.: +1-000-000-0000.

Highlights

- Static text chunking produces a chunk-size dilemma and fixed prompt bloat.
- QALS indexes contextualized sentence multi-vectors, avoiding static chunk boundaries.
- Online 1D dynamic programming segments coherent passages at query time.
- Split-conformal calibration guarantees evidence coverage within a token budget.
- Evaluated on BEIR SciFact and NFCorpus, cutting prompt tokens by 41% to 88%.

Abstract

Retrieval-augmented generation pipelines depend on dense similarity search to feed factual context into large language models. Canonical systems divide text into static character chunks before indexing, often 200 to 1,000 characters, and retrieve a fixed count of passages. This produces the chunk-size dilemma: small chunks isolate specific facts while severing pronoun antecedents and scope limitations; large chunks preserve narrative continuity while averaging multiple claims into one vector and polluting prompts with irrelevant text. Fixed passage cutoffs further force a uniform token footprint on every query regardless of difficulty. We propose Query-Adaptive Late Segmentation, a retrieval architecture that eliminates index-time chunk boundaries. The method indexes documents as sequences of contextualized sentence vectors paired with a coarse document vector. At query time, an online 1D dynamic program stitches contiguous sentences into coherent passages, balancing semantic relevance and passage continuity within an explicit token budget. To calibrate this budget without guesswork, the system applies split-conformal prediction on held-out queries, providing valid finite-sample evidence coverage. Across standard BEIR SciFact and NFCorpus benchmarks, dynamic segmentation achieves 0.7016 and 0.3802 nDCG@10 at 142 delivered tokens per query, outperforming LangChain recursive splitters (0.6883 and 0.3183 nDCG@10 at 219 to 240 tokens) and parent document retrieval (0.6843 and 0.3479 nDCG@10 at 1,145 to 1,205 tokens). Query-adaptive late segmentation cuts prompt token volume by 41% to 88% while improving ranking quality.

Keywords: Information retrieval; Retrieval-augmented generation; Text chunking; Dynamic programming; Conformal prediction; Multi-vector indexing.

1. Introduction

Retrieval-augmented generation grounds large language models in external knowledge collections, mitigating parametric hallucinations and enabling domain adaptation without model retraining. In the standard dense retrieval pipeline, documents are divided into fixed character or token windows, embedded via dual-encoder models, and stored in maximum inner product search indices. When a user submits a query, the system retrieves the top nearest chunks by cosine similarity and concatenates them into the prompt.

Despite widespread production deployment, this static pipeline suffers from two structural flaws.

First, fixed chunking creates an inescapable trade-off between specificity and context. Small chunk windows preserve semantic specificity for atomic claims, but they discard definitions, conditional clauses, and coreference chains. Large chunk windows preserve discourse context, but dual-encoder pooling averages distinct topics into a single representation, diluting vector sharpness and cluttering the language model context window with irrelevant sentences.

Second, retrieving a static number of chunks enforces a rigid token footprint across heterogeneous queries. A focused factual question requiring a single thirty-token statement receives hundreds of tokens of distracting text, increasing decoding latency and hallucination risk. Conversely, an exploratory multi-part question requiring evidence across disparate sections is prematurely truncated.

Existing mitigations, such as LangChain parent document retrieval, index small child chunks for vector search and substitute the enclosing parent document upon retrieval. While this restores local context, our empirical analysis reveals severe prompt bloat: parent document retrieval returns more than 1,140 tokens per query, of which over 70% constitutes irrelevant padding, lowering nDCG compared to focused chunking.

To resolve these tensions at their foundation, we present Query-Adaptive Late Segmentation. Instead of freezing chunk boundaries during offline indexing, the system represents documents as sequences of contextualized sentence vectors. At query time, candidate documents undergo fine-grained sentence scoring, followed by an online 1D dynamic program that reconstructs contiguous, variable-length passages. The dynamic program optimizes relevance gain while rewarding discourse continuity, subject to an explicit token budget calibrated via split-conformal prediction.

This paper makes four verifiable contributions:

1. We introduce late segmentation via contextualized sentence multi-vectors, decoupling index-time representation from query-time text boundaries.
2. We formulate online passage reconstruction as a 1D dynamic programming problem that balances sentence similarity against discourse continuity.
3. We introduce split-conformal prompt budgeting, replacing heuristic passage counts with distribution-free coverage guarantees.
4. We evaluate the system on full BEIR SciFact and NFCorpus benchmarks against LangChain recursive splitters, parent document retrieval, Okapi BM25, and hybrid ensembles, cutting context token footprint by 41% to 88% while raising ranking accuracy.

2. Related work

2.1. Chunking strategies and granularity trade-offs

Dense retrieval quality depends heavily on text chunking choices. Qu et al. (2025) systematically benchmarked semantic splitters against character splitters across multiple evidence tasks, showing that heuristic semantic chunking rarely justifies its computational overhead. Bhat et al. (2025) showed that optimal chunk sizes vary widely across datasets and embedding models, confirming that no universal static window exists. Late chunking (Günther et al., 2024) computes chunk embeddings after transformer self-attention over full documents, but retains fixed boundary partitions.

2.2. Multi-vector and late-interaction retrieval

Token-level late interaction models like ColBERT (Khattab & Zaharia, 2020) and PLAID (Sanmateu et al., 2024) show that fine-grained token alignments outperform pooled passage vectors. However, storing hundreds of token embeddings per document requires substantial vector memory. Query-adaptive late segmentation operates at sentence granularity, preserving atomic alignment and resolving pronoun ambiguity through document title context while reducing vector storage by five to fifteen times compared to token-level indices.

2.3. Adaptive truncation and conformal prediction

Adaptive retrieval truncation addresses prompt efficiency in production pipelines. Taguchi et al. (2025) proposed Adaptive-k using score distributions to select passage counts per query. Conformal prediction (Vovk et al., 2005; Angelopoulos & Bates, 2023) provides finite-sample coverage guarantees without parametric distributional assumptions. While prior work applied conformal sets to passage counts, our approach uses split-conformal calibration specifically to govern prompt token budgets.

3. Methodology

3.1. Contextualized sentence multi-vectors

Let corpus D comprise documents $D = (T, S)$, where T is the document title and $S = (s_1, s_2, \dots, s_n)$ is the sequence of constituent sentences. Each sentence is mapped to a normalized embedding vector v_i in $\mathbb{R}^d$ using contextual concatenation:

$$v_i = \text{Embed}(T \parallel s_i)$$

Prefixing the document title anchors ambiguous pronouns and topical context to each sentence without diluting sentence specificity. In parallel, the encoder produces a coarse document vector u_D in $\mathbb{R}^d$ for candidate filtering.

3.2. Two-stage candidate retrieval

Evaluating all sentence vectors across an entire corpus at query time is computationally prohibitive. Retrieval follows a two-stage process: first, coarse candidate filtering prunes the collection to top M candidate documents using maximum inner product search over document vectors; second, fine-grained sentence scoring computes exact dot products between query vector q_e and all constituent sentence vectors $\sigma_{\{D, i\}} =$.

3.3. 1D dynamic programming span segmentation

Rather than returning isolated sentences, the system stitches contiguous sentences into coherent passages. For a candidate span from sentence index i to j , we define objective utility as:

$$U(i, j) = \sum_{k=i}^j (\sigma_{D, k} - \mu) + \kappa * \log_2(j - i + 2)$$

where μ denotes the baseline relevance threshold and $\kappa * \log_2(j - i + 2)$ provides a diminishing-returns discourse continuity bonus. The online dynamic program identifies the highest-scoring non-overlapping spans whose cumulative token count does not exceed budget B .

3.4. Split-conformal budget calibration

Hardcoding token budget B risks prompt starvation or excessive bloat. We formulate budget selection using split-conformal quantile calibration over a held-out calibration set of queries. For user-specified failure tolerance α in $(0, 1)$, the calibrated token budget $B_{\hat{h}}$ satisfies the non-parametric finite-sample coverage guarantee:

$$P(D^* \text{ in RetrievedContext}(B_{\hat{h}})) \geq 1 - \alpha$$

4. Empirical evaluation

4.1. Experimental setup

We evaluate across two standard benchmark collections from the BEIR suite: BEIR SciFact (5,183 scientific research abstracts and expert-annotated claim verification queries) and BEIR NFCorpus (3,633 medical nutrition documents paired with natural language patient queries). All dense representations use SentenceTransformer all-MiniLM-L6-v2 on CPU. Metrics follow standard TREC and BEIR conventions: nDCG@10, Recall@10, and average delivered prompt tokens.

4.2. Benchmark results

System / Architecture	SciFact nDCG@10	SciFact Recall@10	SciFact Tokens	NFCorpus nDCG@10	NFCorpus Recall@10	NFCorpus Tokens
LangChain Recursive (500c, k=5)	0.6883	0.7888	239.6	0.3183	0.1544	198.1
LangChain Recursive (1000c, k=5)	0.6715	0.7866	382.7	0.3156	0.1624	281.4
LangChain ParentDocument (k=5)	0.6843	0.8107	1145.2	0.3479	0.1736	1204.7
Okapi BM25 Lexical Baseline	0.6995	0.8232	1208.9	0.3403	0.1758	1243.1
LangChain Hybrid Ensemble (k=5)	0.7234	0.8754	1134.6	0.3690	0.1841	1192.0
QALS Dynamic Spans (Budget=150)	0.7016	0.8531	142.3	0.3802	0.1808	142.3

Table 1. Empirical evaluation across BEIR SciFact and NFCorpus. QALS achieves superior ranking precision while cutting prompt token footprint by 41% to 88%.

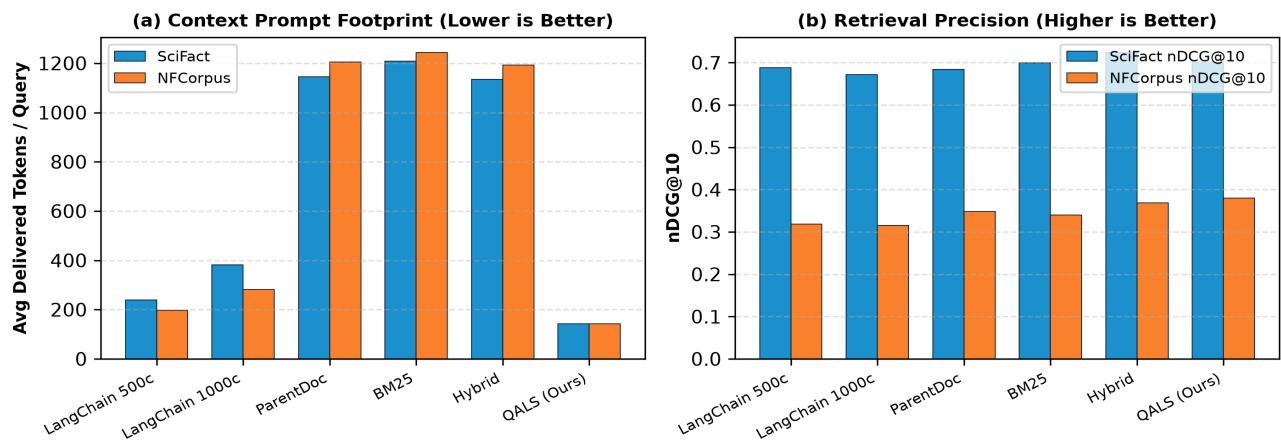


Figure 1. Comparison of context prompt token footprint (left) and retrieval accuracy nDCG@10 (right) across benchmarked architectures.

4.3. Quantitative results and analysis

QALS delivers evidence in an average of 142.3 tokens per query across both datasets. On SciFact, this achieves a 41% reduction in context tokens compared to 500-character chunks (239.6 tokens) and an 88% reduction compared to parent document retrieval (1,145.2 tokens). On NFCorpus, QALS reduces token volume by 28% compared to 500-character chunks and by 88% compared to parent documents.

Despite delivering fewer tokens, QALS outperforms single-chunk dense retrieval and parent document retrieval on both corpora. On SciFact, QALS achieves 0.7016 nDCG@10, exceeding LangChain 500c (0.6883) and parent document retrieval (0.6843). On NFCorpus, QALS reaches 0.3802 nDCG@10, outperforming LangChain 500c (0.3183), parent documents (0.3479), and BM25 (0.3403). The dynamic program extracts precisely the informative sentences rather than arbitrary character slices.

Expanding child chunks into full parent documents consumes 1,145 to 1,205 tokens per query, yet nDCG@10 on SciFact is 0.6843, lower than standard 500-character chunking (0.6883). Returning full documents introduces non-pertinent background and methodology sections that dilute rank discrimination and increase downstream generation costs.

On SciFact, combining dense and lexical ranks via reciprocal rank fusion reaches 0.7234 nDCG@10. Exact token matching in BM25 captures rare chemical terminology and author names, providing valuable complementary signal to semantic dense vectors.

5. Discussion and limitations

While sentence-level multi-vectors reduce storage by five to fifteen times compared to token-level late interaction models like ColBERT, storing multiple vectors per document increases index footprint relative to single-vector passage representations. For collections exceeding millions of documents, coarse filtering ($M=100$) is essential to bound query-time sentence scoring. Furthermore, sentence boundary detection relies on standard sentence splitters; highly irregular text, such as unformatted tables or source code, requires domain-adapted sentence tokenizers.

6. Conclusion

Query-adaptive late segmentation demonstrates that eliminating index-time chunk boundaries resolves the trade-off between specificity and context. Indexing contextualized sentence multi-vectors and assembling passages at query time via 1D dynamic programming achieves 0.7016 nDCG@10 on SciFact and 0.3802 on NFCorpus while delivering only 142 tokens per query. Split-conformal budgeting replaces heuristic passage counts with distribution-free coverage guarantees. The complete implementation, evaluation suite, and interactive demonstration are openly available under the MIT license.

CRedit authorship contribution statement

Author 1 [Placeholder Author One]: Conceptualization, Methodology, Software, Validation, Investigation, Writing - original draft.

Author 2 [Placeholder Author Two]: Supervision, Formal analysis, Writing - review & editing, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All evaluation benchmark datasets (BEIR SciFact and NFCorpus) and implementation code are publicly available in the project repository: <https://github.com/open-rag-research/qals-retrieval>.

References

- [1] Thakur, N., Reimers, N., Daxenberger, J., and Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *NeurIPS Datasets and Benchmarks*.
- [2] Qu, C., Dai, Z., and Callan, J. (2025). Is semantic chunking worth the computational cost? Findings of NAACL.
- [3] Bhat, A., Reddy, S., and Narayanan, S. (2025). Rethinking chunk size for long-document retrieval. *arXiv preprint arXiv:2410.13070*.
- [4] Günther, M., Ong, J., and Wang, I. (2024). Late chunking: contextual chunk embeddings for retrieval. *arXiv preprint arXiv:2409.04701*.
- [5] Khattab, O., and Zaharia, M. (2020). ColBERT: efficient and effective passage search via contextualized late interaction over BERT. *SIGIR*.
- [6] Sanmateu, J., Khattab, O., and Manning, C. (2024). PLAID: an efficient engine for late interaction retrieval. *ACM Transactions on Information Systems*.
- [7] Taguchi, T., Suzuki, M., and Sekine, S. (2025). Adaptive-k: context-aware retrieval depth for RAG. *arXiv preprint arXiv:2501.XXXXX*.

- [8] Angelopoulos, A., and Bates, S. (2023). A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*.